BITWISE OPERATIONS

For Your C File System Project

Block bitmaps, inode bitmaps, flags, masks, and practical C examples

Purpose: This guide explains only the bitwise operations that are useful for your File System project, especially for free-space bitmaps and inode allocation. Every operator is shown with binary examples and practical C code.

1. Why Bitwise Operations Matter in Your Project

Your File System needs compact ways to remember which disk blocks and inodes are free or used. A bitmap does this efficiently: one bit can represent one block or one inode.

```
Example bitmap byte:  00101101
Bit value:            1 = USED, 0 = FREE
Blocks represented:   7 6 5 4 3 2 1 0
```

- Block bitmap: tracks free and occupied disk blocks.
- Inode bitmap: tracks free and occupied inode entries.
- Flags: store multiple yes/no states inside one integer.
- Masks: select, set, clear, or test specific bits.

2. The Bitwise Operators You Need

Operator	Name	Main use in your project	Example
&	AND	Test a bit; apply a mask	value & mask
|	OR	Set a bit to 1	value | mask
^	XOR	Toggle a bit	value ^ mask
~	NOT	Invert bits; useful when clearing	~mask
<<	Left shift	Create a bit mask	1u << bit
>>	Right shift	Move a target bit for inspection	value >> bit

3. Bitwise AND (&) - Test Whether a Bit Is Set

AND returns 1 only when both compared bits are 1. In your project, it is mainly used to check whether a particular block or inode is already marked as used.

```
A = 00101101
B = 00000100
    --------
A & B = 00000100
```

```
unsigned char bitmap = 0b00101101;
unsigned char mask   = 1u << 2;   // 00000100

if (bitmap & mask) {
    printf("Block is USED\n");
} else {
    printf("Block is FREE\n");
}
```

Project use: Use AND in functions such as block_is_used(), inode_is_used(), and when checking status flags.

4. Bitwise OR (|) - Mark a Block or Inode as Used

OR sets a target bit to 1 while leaving the other bits unchanged.

```
bitmap = 00100001
mask   = 00000100
         --------
result = 00100101
```

```
bitmap |= (1u << bit_index);   // mark that bit as USED
```

```
void set_bit(unsigned char *bitmap, int bit_index) {
    bitmap[bit_index / 8] |= (1u << (bit_index % 8));
}
```

Project use: After block_allocate() finds a free block, OR is used to mark its bitmap bit as 1.

5. Bitwise XOR (^) - Toggle a Bit

XOR flips the selected bit: 0 becomes 1 and 1 becomes 0. It is useful for debugging or deliberate toggling, but it is usually not the safest choice for normal allocation because you normally want an explicit set or clear operation.

```
bitmap = 00100101
mask   = 00000100
         --------
result = 00100001
```

```
bitmap ^= (1u << bit_index);   // toggle one bit
```

Project use: Useful for test programs or diagnostic commands. For real allocation logic, prefer OR to set and AND with NOT to clear.

6. Bitwise NOT (~) - Build a Clear Mask

NOT inverts every bit. In your File System, you normally use it together with AND when freeing a block or inode.

```
mask       = 00000100
~mask      = 11111011
bitmap     = 00100101
bitmap&~m  = 00100001
```

```
bitmap &= ~(1u << bit_index);  // clear target bit to 0
```

Project use: When file_delete() releases blocks, clear their bits so block_allocate() can reuse them later.

7. Left Shift (<<) - Create a Bit Mask

Left shift moves bits to the left. The most important pattern in your project is 1 << n, which creates a mask containing one 1-bit at position n.

```
1 << 0  = 00000001
1 << 1  = 00000010
1 << 2  = 00000100
1 << 5  = 00100000
```

```
unsigned int mask = 1u << bit_position;
```

Project use: Every set/test/clear operation on a bitmap needs a mask. Left shift creates that mask.

8. Right Shift (>>) - Move a Bit for Reading

Right shift moves bits to the right. It can be used to move a target bit to position 0 before testing it.

```
value = 00101101
value >> 3 = 00000101
(value >> 3) & 1 = 1
```

```
int bit = (bitmap_byte >> bit_position) & 1u;
```

Project use: This is an alternative way to read one bit. Direct masking with AND is often simpler.

9. The Four Bitmap Patterns You Should Memorize

Task	C expression	Meaning
Test bit	value & (1u << n)	Is bit n currently 1?
Set bit	value |= (1u << n)	Mark bit n as used
Clear bit	value &= ~(1u << n)	Mark bit n as free
Toggle bit	value ^= (1u << n)	Flip bit n

10. Real Block Bitmap Implementation

If your disk has 2048 blocks, you do not need 2048 integers. One byte stores 8 block states, so the bitmap needs only 2048 / 8 = 256 bytes.

```
#define TOTAL_BLOCKS 2048
#define BITS_PER_BYTE 8
#define BITMAP_SIZE ((TOTAL_BLOCKS + 7) / 8)

unsigned char block_bitmap[BITMAP_SIZE];
```

Finding the byte and bit

```
int byte_index = block_number / 8;
int bit_index  = block_number % 8;
```

```
Example: block_number = 13
byte_index = 13 / 8 = 1
bit_index  = 13 % 8 = 5
So block 13 is stored in bit 5 of bitmap byte 1.
```

11. Reusable Bitmap Functions

```
int bitmap_test(const unsigned char *bitmap, int index) {
    int byte_index = index / 8;
    int bit_index  = index % 8;
    return (bitmap[byte_index] & (1u << bit_index)) != 0;
}

void bitmap_set(unsigned char *bitmap, int index) {
    int byte_index = index / 8;
    int bit_index  = index % 8;
    bitmap[byte_index] |= (1u << bit_index);
}

void bitmap_clear(unsigned char *bitmap, int index) {
    int byte_index = index / 8;
    int bit_index  = index % 8;
```

```
    bitmap[byte_index] &= ~(1u << bit_index);
}
```

12. Block Allocation Algorithm Using Bitwise Operations

A simple allocator scans the bitmap from the first data block until it finds a 0 bit. It then sets that bit to 1 and returns the block number.

```
int block_allocate(void) {
    for (int block = DATA_BLOCK_START; block < TOTAL_BLOCKS; block++) {
        if (!bitmap_test(block_bitmap, block)) {
            bitmap_set(block_bitmap, block);
            return block;
        }
    }
    return -1;   // no free block
}
```

Freeing a block

```
void block_free(int block) {
    bitmap_clear(block_bitmap, block);
}
```

Algorithm: This is a linear first-free search. The bitwise operations make the representation compact; the loop decides which free bit to choose.

13. Inode Bitmap - Same Idea, Different Resource

You can use exactly the same helper functions for an inode bitmap. One bit represents one inode entry.

```
unsigned char inode_bitmap[INODE_BITMAP_SIZE];

int inode_allocate(void) {
    for (int i = 0; i < TOTAL_INODES; i++) {
        if (!bitmap_test(inode_bitmap, i)) {
            bitmap_set(inode_bitmap, i);
            return i;
        }
    }
    return -1;
}
```

14. Bit Flags Inside an Inode or Metadata Field

Bitwise operations can also store several yes/no properties inside one integer. This is optional for your first version, but useful if you later add flags.

```
#define FLAG_USED       (1u << 0)
#define FLAG_DIRECTORY  (1u << 1)
#define FLAG_HIDDEN     (1u << 2)

unsigned int flags = 0;
flags |= FLAG_USED;              // set
flags |= FLAG_DIRECTORY;         // set

if (flags & FLAG_DIRECTORY) {
    printf("This inode is a directory\n");
}

flags &= ~FLAG_HIDDEN;           // clear hidden flag
```

Recommendation: For your 7-day project, use bitmaps first. Add packed flags only if the core filesystem is already stable.

15. Important C Pitfalls

- Prefer unsigned types for bit operations, such as unsigned char, unsigned int, uint8_t, or uint32_t.
- Use 1u << n instead of 1 << n when possible; the u makes the constant unsigned.
- Do not shift by a value greater than or equal to the width of the type.
- Remember operator precedence. Write parentheses: bitmap & (1u << n).
- A bitmap bit is not the block itself; it only records the block state.
- When deleting a file, clear every data-block bit that belonged to the file and also clear its inode bit.
- Do not use XOR to free or allocate normal resources; toggling can accidentally reverse the wrong state.

16. Where Each Operation Appears in Your File System

Project feature	Bitwise operation	Typical use
Block Allocation	&, |, ~, <<	test/set/clear block bitmap bits
Inode Management	&, |, ~, <<	test/set/clear inode bitmap bits
File Deletion	~, &, <<	clear block and inode bits
File Creation	&, |, <<	find and mark free inode/blocks
Metadata Flags (optional)	&, |, ~, ^, <<	store and inspect multiple flags
Debugging bitmap	>>, &	read individual bit values

17. Worked Example: Create and Delete a File

Suppose block 10 is free. Its bitmap bit is 0.

```
// Allocate block 10
if (!(block_bitmap[10 / 8] & (1u << (10 % 8)))) {
    block_bitmap[10 / 8] |= (1u << (10 % 8));
}
```

Later, when the file is deleted:

```
// Free block 10
block_bitmap[10 / 8] &= ~(1u << (10 % 8));
```

This is the exact set/test/clear pattern that your block allocator will repeatedly use.

18. Quick Practice

- If bitmap = 00010000, is bit 4 used? Test it with bitmap & (1u << 4).
- Write the expression that marks bit 6 as used.
- Write the expression that frees bit 2.
- For block 29, calculate byte_index = 29 / 8 and bit_index = 29 % 8.
- Explain why a 2048-block bitmap needs only 256 bytes.

19. Final Cheat Sheet

```
// TEST
if (bitmap[byte] & (1u << bit)) { /* used */ }

// SET / ALLOCATE
bitmap[byte] |= (1u << bit);

// CLEAR / FREE
bitmap[byte] &= ~(1u << bit);

// TOGGLE
bitmap[byte] ^= (1u << bit);

// READ AS 0 OR 1
int value = (bitmap[byte] >> bit) & 1u;
```

Most important: For your project, memorize TEST, SET, and CLEAR. Those three patterns are enough to implement block and inode bitmaps correctly.